

Testing, Debugging, and Deployment

Apply advanced techniques and tools for testing Apex classes and triggers, such as mocks and stubs.

Table of Contents

-  [Apex Unit Tests](#)
-  [Apex Class Testing Example](#)
-  [Apex Trigger Testing Example](#)
-  [Error Methods in Apex Testing](#)
-  [Test Setup Method](#)
-  [Test Utility Class](#)
-  [Loading Static Resource](#)
-  [Using Mock Callouts](#)



Table of Contents

-  [Asynchronous Apex Testing](#)
-  [Stubbing in Testing](#)
-  [Testing Flows with External Services](#)
-  [Managed Packages and Unlocked Packages](#)
-  [Best Practices](#)



After studying this topic, you should be able to:

-  Describe the options available for executing Apex unit tests.
-  Explain how to create test classes for Apex classes and triggers.
-  Describe the techniques for loading test data in unit tests.
-  Describe what a mock is and how to use it in testing callouts.
-  Describe what a stub is and how it can be used in unit tests.



Introduction

This topic discusses how to create and execute **unit test classes** for testing **Apex classes** and **triggers**. Test classes and methods are always annotated with the **@isTest** annotation. Unit tests should be created to achieve the **75% minimum code coverage** required for deployment, but developers should write test methods for all possible test cases. Test data can be created for testing. A method annotated with **@testSetup** provides an easy way to create **test data** for all methods in a single test class. Test utility classes and test data stored as **static resources** allow for easy reuse of test data across test classes. **Web service callouts** cannot be tested directly, but Salesforce offers **mock interfaces**, which can be used to simulate test callouts.



Testing Apex Classes & Triggers

Apex Classes



A unit test class can be created for testing an Apex class. The test class and its methods should be annotated with `@isTest`.

Apex Triggers



A unit test class can be created for testing an Apex trigger. It should perform the actions that fire the trigger and verify the expected results.

Apex Test Execution



All or specific methods of a unit test class can be executed. Tests can be executed from various places like the Developer Console and Setup.

Apex Unit Tests

Table of Contents



Apex Unit Tests

Unit tests can be written and executed for **Apex classes** and **triggers** to promote **high-quality** code and **meet** deployment requirements.



MEET EXPECTATIONS

Unit tests ensure that Apex classes and triggers **work** as **expected**.



MEET REQUIREMENTS

For deployment, **at least 75%** of code must be covered by unit tests that pass.



USE CASE TESTING

Unit tests should test the common use cases in the app, including **positive** and **negative** test cases, and **bulk** and **single-record** processing.

Executing Apex Unit Tests

Salesforce **allows** and **provides** different **tools** for executing Apex unit tests. Apex test methods can be executed in the following:



Developer Console



Setup



Salesforce Extensions
for Visual Studio Code



Code Builder



API

Executing Apex Unit Tests

Salesforce **allows** and **provides** different **tools** for executing Apex unit tests. **Apex test methods** can be **executed** in the following ways:



❖ ONE CLASS

All or a subset of test methods in a **specific class**



❖ MULTIPLE CLASSES

All or a subset of test methods in a **set of classes**



❖ TEST SUITE

A predefined **suite** of **test classes**, or also known as a test suite

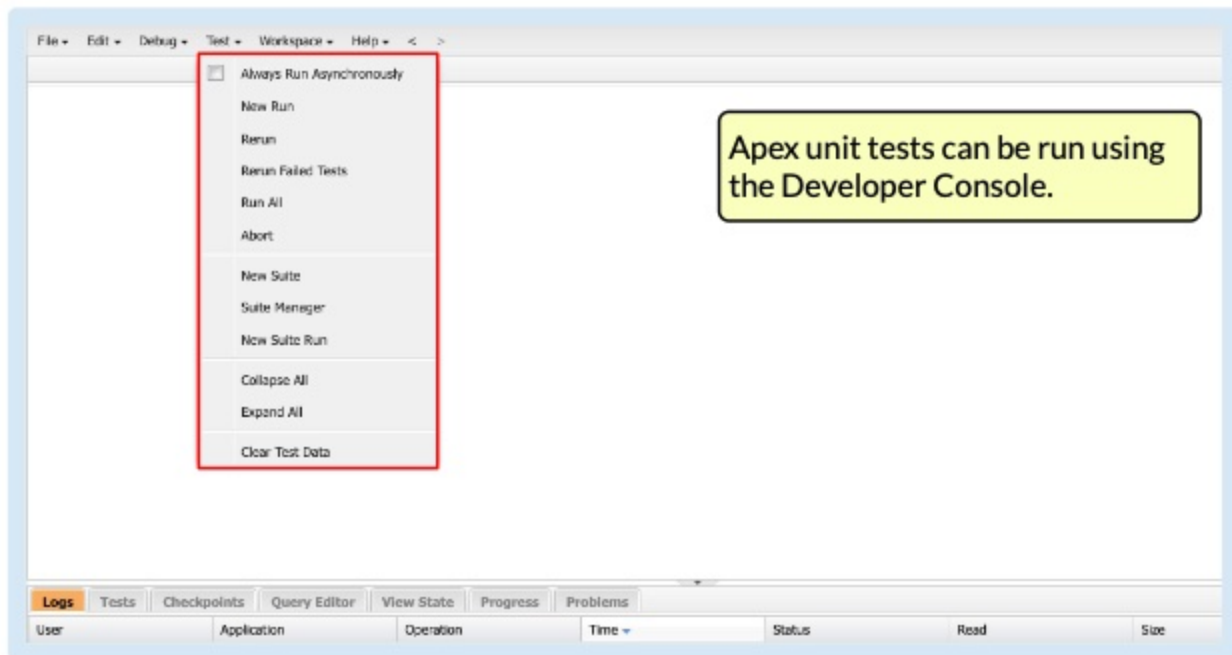


❖ ALL TEST CLASSES

All **unit test** classes in the **org**

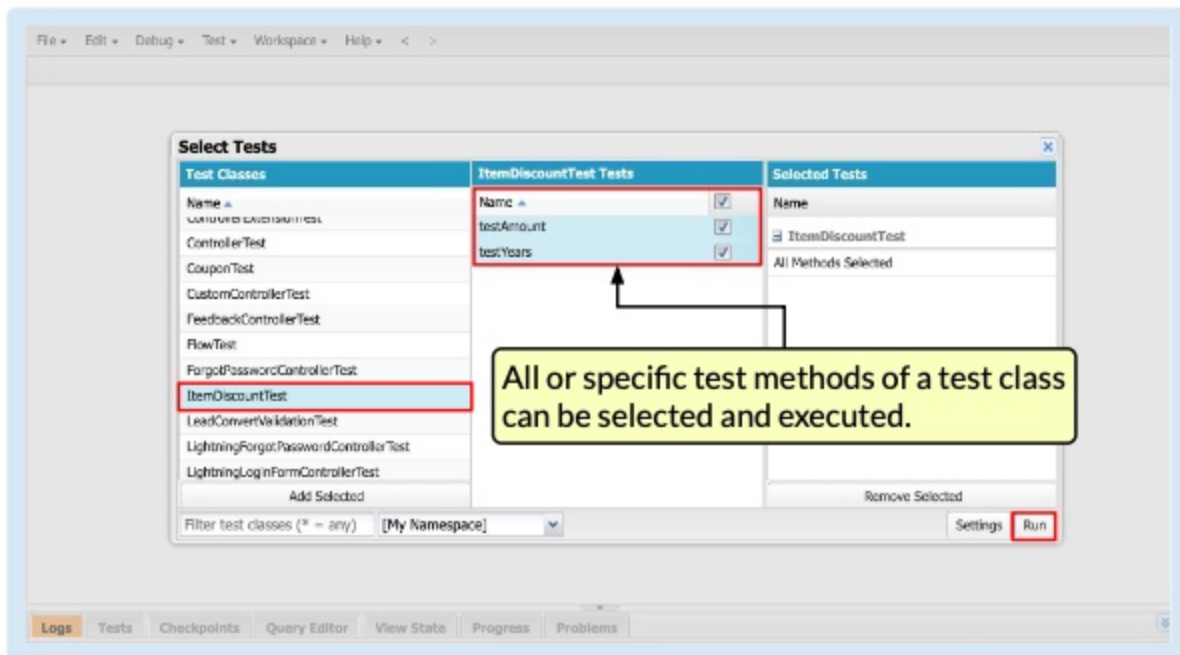
Using the Developer Console

The Developer Console provides **several options** and **settings** for running **Apex tests**.



Executing Unit Test Methods in the Developer Console

Developer Console allows **specifying** the **test methods** to run in a test class.



Creating Unit Tests for Apex Classes

A **separate** Apex class, called a **test class**, is required in order to test a particular Apex class.



CLASS ANNOTATION

The test class should be annotated with **@isTest** and be either **private** or **public**. **Public** test classes are generally used for **test data factory** classes.



METHOD ANNOTATION

Each test method in the test class should be annotated with **@isTest** annotation and take **no arguments**.

Creating Unit Tests for Apex Classes



◆ SETUP METHOD

A test setup method, which can be used for creating **test data**, can be defined using the **@testSetup** annotation.

◆ ACCESS MODIFIER

Access modifiers can be **omitted** from test methods since they are **always accessible** by the testing framework.

Writing Unit Tests

A unit test invoke the **method** or **trigger** that is being tested. If a method or trigger is not called, it will also negatively impact the **code coverage computation**.



TESTING METHODS

The method that is being tested can be **called directly** from the test method.



TESTING TRIGGERS

To test triggers, include **DML operations** in test methods to invoke the triggers.



RUNNING TESTS

Unit tests should **execute** the Apex methods or triggers that are being tested.



VERIFYING RESULTS

Test results should be verified with the use of **assert methods**.

Verifying Code Behavior

Assert methods are used to verify code behavior. These methods throw an **exception** when the **actual** result does not match the **expected** result.

```
// System.assert(condition, msg);  
System.assert(x == y, 'Oops, X does NOT equal Y');  
  
// System.assertEquals(expected, actual, msg);  
System.assertEquals(x, y, 'Oops, X does NOT equal Y');  
  
// System.assertNotEquals(expected, actual, msg);  
System.assertNotEquals(x, y, 'Oops, X does equal Y');
```

```
// Assert.isTrue(condition, msg);  
Assert.isTrue(x == y, 'Oops, X does NOT equal Y');  
  
// Assert.isFalse(condition, msg);  
Assert.isFalse(x == y, 'Oops, X does equal Y');  
  
// Assert.areEqual(expected, actual, msg);  
Assert.areEqual(x, y, 'Oops, X does NOT equal Y');  
  
// Assert.areNotEqual(expected, actual, msg);  
Assert.areNotEqual(x, y, 'Oops, X does equal Y');
```

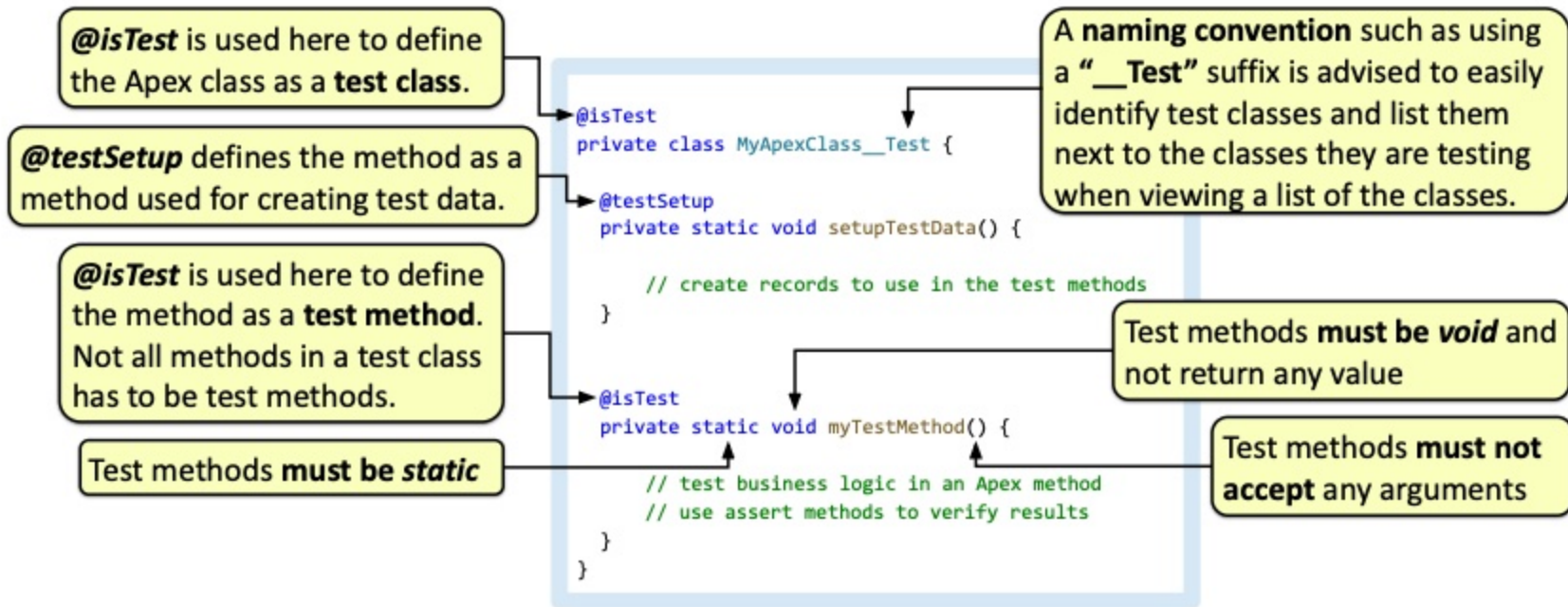
The **System** class provides **assert methods** that can be used for verifying results.

The **Assert** class also provides **assert methods** and **other additional methods** that can be used to help verify code such as testing for **instance types**, **null values**, and explicitly **throwing errors**.

Note that Salesforce **recommends** using the **Assert** class for writing assertions in Apex tests, as it enhances **flexibility** and improves **clarity**.

Anatomy of an Apex Test Class

Below shows a **sample typical structure** of an Apex test class.



Apex Class Testing Example

Table of Contents



Example of Testing an Apex Class

This Apex class **creates** a sales order and **updates** a field for every opportunity that is **closed won**.

```
public class OpportunityUpdate {  
  
    public static void processRecords (List<Opportunity> opps) {  
  
        List<Sales_Order__c> salesOrders = new List<Sales_Order__c>();  
        List<Opportunity> oppsToUpdate = new List<Opportunity>();  
  
        for (Opportunity o : opps) {  
            if (o.StageName == 'Closed Won') {  
                Sales_Order__c s = new Sales_Order__c();  
                s.Name = o.Name + ' Order';  
                s.Opportunity__c = o.Id;  
                s.Status__c = 'New';  
                s.Account__c = o.AccountId;  
                salesOrders.add(s);  
  
                o.Notes__c = 'A sales order has been created for this opportunity.';  
                oppsToUpdate.add(o);  
            }  
        }  
  
        insert salesOrders;  
        update oppsToUpdate;  
    }  
}
```

This is an Apex class that needs to be tested.

Example of Testing an Apex Class

This **test class** contains **test methods** for testing the *OpportunityUpdate* Apex class **example**.

```
@isTest
public class TestOpportunityUpdate {

    @testSetup // This method is used to set up test data for testing.
    static void setup() {

        Account a = new Account(Name = 'Test Account');
        insert a;
        List<Opportunity> testOpportunities = new List<Opportunity>();

        for (Integer i=1; i<=300; i++) {
            testOpportunities.add(new Opportunity(
                Name = 'Test Opportunity ' + i,
                StageName = 'Closed Won',
                CloseDate = System.today(),
                Notes__c = '',
                AccountId = a.Id
            ));
        }
        insert testOpportunities;
    }
}
```

This is a test class that tests the *OpportunityUpdate* Apex class.

Continued...

Example of Testing an Apex Class

```
// This test method verifies whether a sales order is created when the stage of an opportunity is changed to 'Closed Won'.
// It also verifies whether the Notes__c field on the opportunity is updated.
@isTest
static void testSalesOrderCreation() {
    List<Opportunity> opps = [SELECT Id, Name, StageName, AccountId, Notes__c FROM Opportunity WHERE StageName = 'Closed Won'];
    OpportunityUpdate.processRecords(opps);
    List<Sales_Order__c> salesOrders = [SELECT Id, Name, Opportunity__c, Status__c, Account__c
    FROM Sales_Order__c WHERE Opportunity__c IN: opps];
    Assert.IsFalse(salesOrders.isEmpty()); // fail test if no sales order were created
    String note = 'A sales order has been created for this opportunity.';
    for (Sales_Order__c so : salesOrders) {
        for (Opportunity o : opps) {
            if (so.Opportunity__c == o.Id) {
                Assert.areEqual(o.Name + ' Order', so.Name); // fail test if the naming convention was not followed
                Assert.areEqual('New', so.Status__c); // fail test if the order status is not 'New'
                Assert.areEqual(o.AccountId, so.Account__c); // fail test if the account IDs do not match
                Assert.areEqual(note, o.Notes__c); // fail test if the note was not saved
            }
        }
    }
}
```

Apex Class Test Coverage

Running the **test class** using the Developer Console shows the test results in the **Tests** tab and coverage details in the **Overall Code Coverage** panel.

The screenshot displays the Salesforce Developer Console interface. The top pane shows the source code for the `OpportunityUpdate` Apex class. The bottom pane shows the **Tests** tab with a list of test runs. A red box highlights the test run `TestOpportunityUpdate`. A yellow callout box points to the test results, stating: "The test passed and covered 100% of the Apex class." To the right, the **Overall Code Coverage** panel shows a table with the following data:

Overall Code Coverage		
Class	Percent	Lines
OpportunityUpdate	100%	15/15
OuterClass	0%	0/19

Apex Trigger Testing Example

Table of Contents



Example of Testing an Apex Trigger

This Apex trigger **creates** a new order when the status of an opportunity is **changed** to 'Closed Won'.

```
trigger OpportunityTrigger on Opportunity (before update) {
    List<Order> newOrders = new List<Order>();

    for (Opportunity opp : Trigger.new) {

        if (opp.StageName == 'Closed Won' &&
            (Trigger.oldMap.get(opp.Id).StageName != 'Closed Won')) {

            Order o = new Order();
            o.AccountId = opp.AccountId;
            o.EffectiveDate = System.today();
            o.Status = 'Draft';
            newOrders.add(o);
        }
    }

    insert newOrders;
}
```

This is an Apex trigger
that needs to be tested.

Example of Testing an Apex Trigger

The Apex **test class** below contains **test methods** for testing the *OpportunityTrigger* trigger **example**.

```
@isTest
public class TestOpportunityTrigger {
    // This method is used to set up test data for testing.
    @testSetup
    static void setup() {
        Account a = new Account(Name = 'Test Account');
        insert a;

        List<Opportunity> testOpportunities = new List<Opportunity>();
        for(Integer i=1; i<=300; i++) {
            testOpportunities.add(new Opportunity (
                Name = 'Test Opportunity ' + i,
                StageName = 'Prospecting',
                CloseDate = System.today(),
                AccountId = a.Id
            ));
        }

        insert testOpportunities;
    }
}
```

This is a test class that tests the *OpportunityTrigger* Apex trigger.

Continued...

Example of Testing an Apex Trigger

```
// This test method verifies whether an order is created when the stage of an opportunity is changed to 'Closed Won'.
@Test
static void testOrderCreation() {
    List<Opportunity> opps = [SELECT Id, Name, StageName, AccountId FROM Opportunity WHERE StageName = 'Prospecting'];
    List<Opportunity> newOpps = new List<Opportunity>();
    for(Opportunity opp : opps) {
        opp.StageName = 'Closed Won';
        newOpps.add(opp);
    }
    update newOpps;

    List<Order> orders = [SELECT Id, Status, EffectiveDate, AccountId FROM Order WHERE AccountId = :opps[0].AccountId];
    Assert.isTrue(orders.size() > 0, 'No orders were found!');

    for (Order o : orders) {
        Assert.areEqual(o.Status, 'Draft');
        Assert.areEqual(o.AccountId, newOpps[0].AccountId);
        Assert.areEqual(o.EffectiveDate, System.today());
    }
}
```

Apex Trigger Test Coverage

Running the **test class** using the Developer Console shows the test results in the **Tests** tab and coverage details in the **Overall Code Coverage** panel.

```
1 //This Apex trigger creates a new order when the status of an opportunity is changed to 'Closed Won'.
2 trigger OpportunityTrigger on Opportunity (before update) {
3
4     List<Order> newOrders = new List<Order>();
5
6     for (Opportunity opp : Trigger.new) {
7         if(opp.StageName == 'Closed Won') {
8             Order o = new Order();
9             o.AccountId = opp.AccountId;
10            o.EffectiveDate = System.today();
11            o.Status = 'Draft';
12            newOrders.add(o);
13        }
14    }
15
16    insert newOrders;
17 }
18
```

The test passed and covered 100% of the Apex trigger.

Status	Test Run	Enqueued Time	Duration	Failures	Total
✓	TestRun @ 3:07:36 pm			0	1
✓	TestRun @ 3:37:48 pm			0	1
✓	TestOpportunityTrigger			0	1
✓	testOrderCreation		0:11		

Overall Code Coverage		
Class	Percent	Lines
OpportunityMaintenance	0%	0/4
OpportunityTrigger	100%	9/9
OpportunityTriggerTest	0%	0/6
OpportunityUpdate	100%	15/15
OrderClass	na	n/a

Error Methods in Apex Testing

Table of Contents



Apex Testing with sObject Error Methods

The *sObject* class contains error methods that can be used in Apex testing to **track** and **obtain** errors without performing any **DML operation**.



sObject.addError()

This method can be used to **dynamically** add errors to specific **fields**.

```
// Example:  
String field = 'Site';  
String msg = 'website does not exist.';  
account.addError(field, msg);
```



sObject.hasErrors()

This method can be used to **determine** if an object instance **contains** any errors.

```
// Example:  
Boolean has = account.hasErrors();  
Assert.areEqual(true, has);
```



sObject.getErrors()

This method can be used to **retrieve** a list of **errors** for a specific object **instance**.

```
// Example:  
Integer size =  
account.getErrors().size();  
Assert.areEqual(1, size);
```

sObject Error Methods Example

The following shows an **example** of using of the **error methods** in testing Apex code.

```
Account a = new Account();
a.Name = 'ACME Inc.';
a.Site = 'https://www.nonworkingwebsite.com';

URLStatus status = (new URLStatus()).check(a.Site); // check if Site URL is accessible (mock)
Assert.areEqual(false, status.healthy);

a.addError('Site', status.message); // add error message to specific field
Assert.areEqual(true, a.hasErrors()); // assert that an message was added to the instance

List<Database.Error> errors = a.getErrors(); // get any errors added to the instance
Assert.areEqual(1, errors.size()); // assert that there should only be 1 error

Database.Error error = errors.get(0);
Assert.areEqual(StatusCode.FIELD_CUSTOM_VALIDATION_EXCEPTION, error.getStatusCode());
Assert.areEqual('Site is not working!', error.getMessage());
Assert.areEqual(1, error.getFields().size());

System.debug('message: ' + error.getMessage());
System.debug('status: ' + error.getStatusCode());
System.debug('fields: ' + error.getFields());
```

Note that errors added via the *addError* method are assigned to this **exception type**, although is not related to custom validation rules.

Execution Log		
Timestamp	Event	Details
23:48:13:021	USER_DEBUG	[32] DEBUG message: Site is not working!
23:48:13:021	USER_DEBUG	[33] DEBUG status: FIELD_CUSTOM_VALIDATION_EXCEPTION
23:48:13:021	USER_DEBUG	[34] DEBUG fields: (Site)

Apex Test Data and Callouts

Salesforce provides the **following features** for working with **data** and **callouts** in unit tests:



TEST SETUP METHOD

A test setup method is annotated with **@testSetup** and used for creating **test data** once for the test methods of the test class.



TEST UTILITY CLASS

A test utility class uses the **@isTest** annotation and creates **test records** that can be used by test classes of the org.



LOADING STATIC RESOURCE

A static resource can be created for a **.csv file** that contains **test records**, and it can be used in a test method using **Test.loadData**.



MOCK REST CALLOUTS

REST callout can be tested using the **HttpCalloutMock** interface; **Test.setMock** is used to set the mock callout in a test method.



MOCK SOAP CALLOUTS

SOAP callout can be tested using the **WebServiceMock** interface; **Test.setMock** is used to set the mock callout in a test method.

Test Setup Method

Table of Contents



Test Setup Methods

A test class is allowed to only have **one** test setup method, which is used for creating **test data** used in the **tests**.



STRUCTURE



Test setup methods are **defined** in a **test class**, take **no arguments**, and **return no value**.



ANNOTATION

Test setup methods are **annotated** using the **@testSetup** annotation.



RUN FIRST



The **test setup method** is run first, **before** any test method in the test class.



TEST DATA



A test setup method **creates data once**, which is **accessible** by all test methods in the test class.



Test Setup Methods



RECORD CREATION

While records created inside a regular **test method** are rolled back at the end of the **method execution**, records created by a **test setup** method are rolled back at the end of **class execution**, which reduces test execution time.



RECORD CHANGES

If a test method changes records created by a test setup method, the changes are rolled back at the end of the method execution, which means that **every** test method **starts** with the **original, unchanged data**.



METHOD SUPPORT

Test setup methods are **not supported** in a test class that uses the **@isTest (SeeAllData=True)** annotation.



Example of a Test Setup Method

The following test class uses a **test setup method** to create **account records** for testing.

```
@isTest
private class AccountTestClass {

    // The @testSetup annotation is used for a test setup method
    @testSetup static void setup() {

        List<Account> testAccounts = new List<Account>();

        for(Integer i=1; i<=300; i++) {
            testAccounts.add(new Account (Name = 'Test Account ' + i));
        }

        insert testAccounts;
    }

    // The test record created in test setup method can be used in a test method
    @isTest
    static void testMethod1() {

        Account acc = [SELECT Id, Name, Industry FROM Account WHERE Name = 'Test Account 1'];
        acc.Industry = 'Healthcare';
        update acc;
    }
}
```

Continued...

Example of a Test Setup Method

```
// Any changes to a test record created in a test setup method are rolled back at the end of a test method
@Test static void testMethod2() {

    Account acc = [SELECT Id, Name, Industry FROM Account WHERE Name = 'Test Account 1'];

    // Verify that the value of the Industry field currently has no value
    Assert.areEqual(null, acc.Industry);

    // The second test method can also make changes to the same test records
    acc.Industry = 'Communication';
    update acc;
}
```

Test Utility Class

Table of Contents



Test Utility Class

When test classes use similar test data, a **test utility class** or **test data factory** can be created and utilized in the different unit tests.

ANNOTATION

A test utility class uses the **@isTest** annotation, which **excludes** the code from the org code size limit and code coverage requirement.

PARAMETERS

Methods in the test utility class can take **parameters** and **return** a **value**.

METHOD CALL

Test creation methods are **called directly from the unit tests**.

METHOD ACCESS

Test creation methods should be defined as **global** or **public** to ensure **unit tests** can access them.

Example of a Test Utility Class

This following utility class creates **account data** for testing, and can be used by a **test class** that requires the same **test data**.

```
// The @isTest annotation is used for the test utility class so that it is executed in test context.
@isTest
public class TestDataFactory {

    // A method in a test utility class can accept parameters and return a value
    public static List<Account> createTestAccounts(String t, String i) {

        List<Account> accounts = new List<Account>();
        for (Integer j = 1; j <= 300; j++) {
            Account a = new Account(Name = 'Test' + j, Type = t, Industry = i);
            accounts.add(a);
        }

        insert accounts;
        return accounts;
    }
}
```


Loading Static Resource

Table of Contents



Load Test Data from Static Resource

Salesforce allows **generating** test data from a static resource.

◆ FILE FORMAT

The **data** that needs to be generated as test records should be added and stored in a **.csv file**.

◆ UPLOAD FILE

The file containing the test data should be uploaded as a **static resource** in the org.

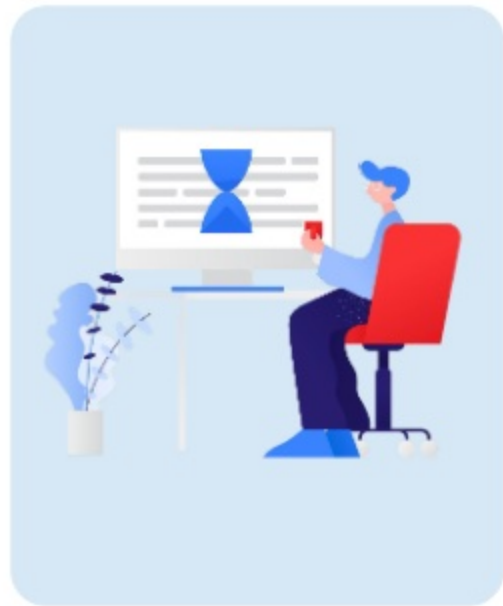
◆ LOAD DATA

Data can be loaded using the **Test.loadData** method by specifying the **object type** and **static resource**.



Load Test Data from Static Resource

There are certain considerations for **creating** the **file** for test data:



◆ PER OBJECT

One file should be used **for each object type** that needs test data.

◆ FIELD NAMES

The **first line** of the file must **contain field names** for the object.

◆ FIELD VALUES

Remaining lines contain **field values** for records to be loaded.

Static Resource Creation

The CSV file needs to be **uploaded** as a static resource so that it can be **referenced** in the test.

The screenshot shows the 'Static Resources' setup page. On the left, a sidebar contains a search bar with 'static' entered, a 'Custom Code' link, and a 'Static Resources' link highlighted with a red box. Below the sidebar, a message states: 'Didn't find what you're looking for? Try using Global Search.' The main content area is titled 'Static Resource' and includes a 'Help for this Page' link. A yellow callout box with the text 'A static resource is created for the CSV file.' has an arrow pointing to the 'Static Resource Information' section. This section is titled 'Static Resource Edit' and contains a 'Save' and 'Cancel' button. Below the title bar, the 'Static Resource Information' section is outlined with a red box and contains the following fields: 'Name' with the value 'testData', 'Description' with an empty text area, 'File' with a 'Choose file' button and the filename 'testData.csv', and 'Cache Control' with a dropdown menu set to 'Private'. At the bottom of the form, there are 'Save' and 'Cancel' buttons.

static

Custom Code

Static Resources

Didn't find what you're looking for? Try using Global Search.

Static Resource

Help for this Page

Static Resource Edit

Save Cancel

Static Resource Information

Name testData

Description

File Choose file testData.csv

Cache Control Private

Save Cancel

A static resource is created for the CSV file.

Example of Loading Test Data from Static Resource

To load test data using the static resource, the **object type** and **name** of the static file is passed to the **Test.loadData** method.

```
// This test class uses Test.loadData to load test data from a CSV file
@Test
private class TestClass {

    @Test
    static void testMethod1() {

        List<Object> accounts = Test.loadData(Account.sObjectType, 'testData');
        Assert.isTrue(accounts.size() == 3);
        Account acc = (Account) accounts[1];

        Assert.areEqual('Hermes', acc.Name);
    }
}
```

Using Mock Callouts

[Table of Contents](#)



Generate REST Callout Mock Response

Salesforce does not allow performing **callouts** in test classes, and instead provides an interface for **simulating** callouts in **test executions**.

	HTTPCALLOUTMOCK HTTP callouts can be tested by creating a class that implements the <i>HttpCalloutMock</i> interface.				
	ACCESS The access modifier of the Apex class should be defined as either global or public.		ANNOTATION The class should be annotated with @isTest annotation, since it is to be used only in test context.		RESPOND The class only needs to implement a respond() method which accepts an <i>HttpRequest</i> and returns an <i>HttpResponse</i>.
	RESPONSE The <i>HttpResponse</i> should mimic what the actual REST-based web service would return.		STANDALONE CLASS As a best practice, a separate class should be created for each tested callout.		MOCK RESPONSE In the respond() method, the mock response can be sent only for a specific endpoint and HTTP method.

Generate REST Callout Mock Response



SET CALLOUT MOCK

In the main unit test, `Test.setMock` method is used to set the mock callout and **receive** the **fake response** from the mock callout class.



FIRST ARGUMENT

`HttpCalloutMock.class` is used as the first argument for `Test.setMock`.



SECOND ARGUMENT

For the second argument, an **instance of the interface implementation** of `HttpCalloutMock` is passed. For example:
`Test.setMock(HttpCalloutMock.class, new MockContactServiceCallout());`

Generate REST Callout Mock Response



VERIFY RESPONSE

The HttpResponseMessage generated by the mock callout can be [verified in the unit test method](#).



SETMOCK POSITION

Test.setMock() should be added [before](#) the callout. Note that if the test class creates test data, an 'uncommitted work' [error](#) will be thrown if the callout is invoked without enclosing it in Test.startTest() and Test.stopTest() methods.

REST Callout Mock Example

The following shows an **example** of a callout mock class and how it is used in a **test method**.

```
// example callout mock class that simulates the response of a successful enrollment of a student:
public class EnrollHttpCalloutMock implements HttpCalloutMock {
    public HttpResponse respond(HTTPRequest req) {
        HttpResponse res = new HttpResponse();
        res.setHeader('Content-Type', 'application/json');
        res.setBody('{"success":true,"studentId":12345,"courseId":67,"yearLevel":3}'); // create fake response
        res.setStatusCode(200);
        return res;
    }
}

// example code contained and executed inside a test method:
Test.startTest();
Test.setMock(HttpCalloutMock.class, new EnrollHttpCalloutMock());
StudentApplicationWrapper application = generateStudentApplication(); // some helper method
Enrollment erm = StudentController.enroll(application); // invoke the method to test

// verify new record created from fake data
Assert.areEqual(erm.ExternalStudentId__c, 12345);
Assert.areEqual(erm.YearLevel__c, 3);
Assert.areEqual(erm.CourseId__c, 67);
Test.stopTest();
```

Generate SOAP Callout Mock Response

SOAP callouts can be tested by creating a class that **implements** the **WebServiceMock** interface based on classes generated by WSDL2Apex.



STANDALONE CLASS

As a best practice, a **separate class** should be created for **each callout** to test.



ACCESS MODIFIER

The implementing Apex class should be defined as either **global** or **public**.



CLASS ANNOTATION

The implementing Apex class is annotated with **@isTest annotation**.

Generate SOAP Callout Mock Response



SINGLE METHOD

The implementing Apex class only needs to implement a **doInvoke()** method.



METHOD PARAMETERS

All **information** about the **callout**, including the response, is passed to doInvoke() through **multiple input parameters**.



MOCK RESPONSE

The response is placed in the **response parameter** of type **Map<String, Object>**.



SPECIFIC ENDPOINT

In the doInvoke() method, a **specific endpoint** and **method** can be used.

Generate SOAP Callout Mock Response

In the main unit test, **Test.setMock** is used to set the mock callout and **receive** the **fake response** from the mock callout class.



FIRST ARGUMENT

WebServiceMock.class is used as the first argument for Test.setMock.



SECOND ARGUMENT

For the second argument, an **instance of the interface implementation** of WebServiceMock.class is passed. For example:

```
Test.setMock(WebServiceMock.class, new MockContactWebService());
```



VERIFY RESPONSE

The response parameter passed to the callout is **populated** and can be **verified in the unit test method**.

Asynchronous Apex Testing

Table of Contents



Asynchronous Apex Testing

Asynchronous Apex is executed in a **different transaction and time**. However, unit tests can be defined in a way that will cause it to run in a **synchronous manner** so that its result can be verified accordingly.



FUTURE METHOD

Executed in a different transaction and only when resources are available



QUEUEABLE APEX

Like future methods, but supports job chaining and other data types



BATCH APEX

Used to perform operations that require higher governor limits



SCHEDULED APEX

Used to run Apex at scheduled intervals or once, by aborting after job completion

Future Method Testing

To test an “*updateByCode*” future method, for example, it is invoked in between the **Test.startTest()** and **Test.stopTest()** methods.

```
@IsTest
private class ProductControllerTest {

    // ...

    @IsTest
    private static void testUpdateByCode() {
        Test.setMock(HttpCalloutMock.class, new ERPCalloutMock());

        Test.startTest();
        ProductController.updateByCode('123ABC'); // future method
        Test.stopTest();

        // verify results of updateByCode method
        List<Product_Log__c> logs = [SELECT Product_Code__c FROM Product_Log__c];
        Assert.areEqual(1, logs.size());
        Assert.areEqual('123ABC', logs[0].Product_Code__c);
    }

    // ...
}
```

After the *Test.startTest()* method, any **asynchronous processes** are collected. When *Test.stopTest()* method is called, any collected processes are **run synchronously**. This then allows any results to be verified later in the **same transaction**.

Testing Queueable Apex and Batch Apex

The following shows how Queueable Apex and Batch Apex are tested in the same manner using **Test.startTest()** and **Test.stopTest()** methods.

Queueable Apex test

```
// ...

@isTest
private static void testAssigner() {

    // get current records assigned to a specific owner
    List<Opportunity> opps = [SELECT Id FROM Opportunity
        WHERE OwnerId = :users.get(0).Id];
    // create a job to assign opportunities to a new owner
    Assigner assignment = new Assigner(opps, users.get(1).Id);

    Test.startTest();
    System.enqueueJob.assignment); // reassign the records
    Test.stopTest();

    // validate results
    List<Opportunity> results = [SELECT Id FROM Opportunity
        WHERE OwnerId = :users.get(0)];
    Assert.areEqual(0, results.size());
}

// ...
```

Batch Apex test

```
// ...

@isTest
private static void testCouponUpdates() {

    Test.startTest();
    // update coupon details on products
    CouponUpdate updates = new CouponUpdate();
    Id batchId = Database.executeBatch(updates);
    Test.stopTest();

    // validate results
    Assert.areEqual(100, [SELECT COUNT() FROM Product__c
        WHERE CouponDate__c = TODAY]);
}

// ...
```

Testing Scheduled Apex

As scheduled Apex is also asynchronous, it is tested using the same approach. In this example, a **schedulable class** is used to create task reminders for sales users with **unread leads**.

```
// ...

// "unreadLeads" = leads that haven't been viewed yet
Map<Id, Lead> leadMap = new Map<Id, Lead>(unreadLeads);
// get the IDs of the leads
List<Id> leadIds = new List<Id>(leadMap.keySet());

Test.startTest();

// job will be executed immediately regardless of the set date due to start and stop methods
String runDate = '0 0 0 14 2 ? 2099'; // Feb 12, 2099

// schedule the schedulable Apex
System.schedule('MyReadLeadsTask', runDate, new ReadLeadReminder());

// no tasks should have been created yet at this point
List<Task> tasks = [SELECT Id FROM Task WHERE WhatId IN :leadIds];
Assert.AreEqual(0, tasks.size(), 'Tasks already exist!');
Test.stopTest();

// tasks should already have been created at this point
tasks = [SELECT Id FROM Task WHERE WhatId IN :leadIds];
Assert.AreEqual(leadIds.size(), tasks.size(), 'Scheduled reminders failed!');
```

Scheduled Apex test

Stubbing in Testing

[Table of Contents](#)



What is Stubbing

While a mock callout is used to return a fake response from a callout, a stub can be used to return a **fake response** from an invoked method by using a **mock version** of the method's class.



TEST ISOLATION

Stubbing is used to prevent **code dependencies** from interfering in a test and allows the unit test to **isolate** the method that is being tested.



STUB CLASS

To mock a class, an **Apex class** that implements the **StubProvider** interface is created. Then, the stub class is instantiated using the **Test.createStub** method. The **System.StubProvider** interface and **System.Test.createStub** method make up the **Apex stub API**.

What is Stubbing

METHOD



The stub class has a method called **handleMethodCall**, which contains several parameters that can be used to **determine the method** that is being invoked in the test and cause the invoked method to, for example, return a custom value.

STUBBING



To implement stubbing, actual code is designed to **allow the stub class** to be used in place of the actual class when testing. For example, a **unit test** is created to test **classA.myMethod**, which is dependent on an output from **classB.myMethod**. To avoid the dependency, a **mock version** of **classB** is created, then code is written to ensure that the mock version is being referenced in the test instead and not the actual class.

How to Use a Stub

The following shows the **high-level steps** of using a stub in a test class.

INSTANTIATE

Instantiate the **stub class** using the **Test.createStub** method.

EXECUTE CODE

Call the **Test.startTest()** method and then **execute the code** that needs to be tested.

ASSERT RESULTS

Call the **Test.stopTest()** method and then **verify the results** using assertions.

Stubbing Example

In this example, a **VehicleDoctor** class is created which contains a method called **checkVehicleStatus**. As shown, this method has a dependency on the **getTotalMilesTraveled** method from a **Vehicle** class

```
public class VehicleDoctor {  
  
    public Boolean checkVehicleStatus(Vehicle v) {  
  
        Boolean status = true;  
        Decimal miles = v.getTotalMilesTraveled();  
  
        if (miles >= 100000.00) {  
            status = false;  
        }  
  
        return status;  
    }  
}
```

```
public class Vehicle {  
  
    public Decimal getTotalMilesTraveled() {  
  
        Decimal distance = 0.00;  
  
        // i.e.: some "complex" logic here to calculate distance  
        // distance = ...  
  
        return distance;  
    }  
}
```


Stubbing Example

A unit test needs to be created to test the **checkVehicleStatus** method. To avoid the dependency on the Vehicle class in the test, a **stub class**, which is shown below, is created to mock the Vehicle class.

```
@isTest
public class VehicleMock implements System.StubProvider {

    Decimal distance;

    public VehicleMock(Decimal d) {
        this.distance = d;
    }

    public Object handleMethodCall(Object stubbedObject, String stubbedMethodName, Type returnType,
        List<Type> listOfParamTypes, List<String> listOfParamNames, List<Object> listOfArgs){

        if (stubbedMethodName == 'getTotalMilesTraveled') {
            System.debug(this.distance);
            return this.distance;
        }

        return null;
    }
}
```

A common practice is to annotate the stub class with **@isTest** so it doesn't require **code coverage** and is **visible only** to test classes.

This example stub class is defined with a **constructor** that is used to **preset** the value of the distance.

When the **getTotalMilesTraveled** method gets called in the unit test, the code in this block gets executed instead.

Stubbing Example

The following **test class** illustrates how the **VehicleMock** stub class is first instantiated and then used in the **checkVehicleStatus** method that is being tested in the unit test.

```
@isTest
private class VehicleDoctorTest {

    @isTest
    static void testCheckVehicleStatus() {

        // instantiate the stub class
        Vehicle vm = (Vehicle) Test.createStub(Vehicle.class, new VehicleMock(100000.00));

        VehicleDoctor vd = new VehicleDoctor();
        Boolean status;

        Test.startTest();
        status = vd.checkVehicleStatus(vm);
        Test.stopTest();

        Assert.areEqual(false, status, 'expected result false');
    }
}
```

The object that is **being stubbed** is passed as the **first parameter** of the *Test.createStub* method.

Since *Test.createStub()* returns an **Object**, the value needs to be **casted back** to the proper **data type**.

In this example, the **distance** is already preset when the stub class is **instantiated**.

Stub Limitations

Stubs are very flexible and can be used to generally mock classes and methods. However, the following specific **Apex elements** cannot be mocked.

Batch Apex classes and
Inner classes

Classes that only have
private constructors

Static methods (including
future methods) and
private methods

Class Properties
(**getters** and **setters**)

Apex **triggers**

System **types**

Testing Flows with External Services

Table of Contents



Testing Flows with External Services

Apex unit tests can also be created for flows that use an **invocable action** with External Services. The **HttpCalloutMock interface** can be used to simulate the callout to the **external service endpoint**.

```
global class StudentPortalHttpCalloutMock implements HttpCalloutMock {
    global HTTPResponse respond(HTTPRequest request) {
        // Assert expected request test data
        Assert.areEqual('GET', request.getMethod());
        Assert.areEqual('callout:StudentPortalService/G6450956Q', request.getEndpoint());
        Assert.areEqual('astro', request.getHeader('accessCode'));

        // Send response test data
        HTTPResponse response = new HTTPResponse();
        response.setHeader('Content-Type', 'application/json');
        response.setBody('{"level": 3, "name": "John Smith", ' +
            '"course": "BSME", "gender": "M", "birth": "20050101"}');
        response.setStatusCode(200);
        return response;
    }
}
```

In this example, the **external service** is registered with a **Named Credential** called *StudentPortalService* which is used to retrieve basic student information.

The **mock implementation** tests the details of the **requests** from the flow and then **responds** with an appropriate HTTP response output.

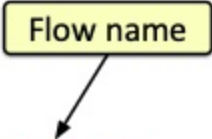
Testing Flows with External Services

The flow test class sets up the **HTTP callout mock** for the external service action, creates and runs the flow interview, and then **asserts the actual flow output parameters** with the expected test values.

```
@IsTest
public class StudentPortalServiceFlowTest {
    @IsTest
    static public void testStudentPortalServiceFlow() {
        // Set HTTP callout mock to match flow's external service action invocation
        Test.setMock(HttpCalloutMock.class, new StudentPortalHttpCalloutMock());
        // Set flow input variables and create the flow interview
        Map<String, Object> inputVariables = new Map<String, Object>();
        Flow.Interview myStudentPortalFlow = Flow.Interview.createInterview('StudentPortalServiceFlow', inputVariables);

        myStudentPortalFlow.start(); // Start flow interview with set input variables

        // Assert flow output variables
        Assert.areEqual('John Smith', (String)myStudentPortalFlow.getVariableValue('fullname'));
        Assert.areEqual('BSME', (String)myStudentPortalFlow.getVariableValue('studentCourse'));
        // ...
    }
}
```



Managed Packages and Unlocked Packages

Table of Contents



Managed Packages and Unlocked Packages

When running the **SFDX report command** to retrieve details of a package version for **second-generation managed** or **unlocked packages**, the output can also include **code coverage details**.

❖ MANAGED PACKAGES

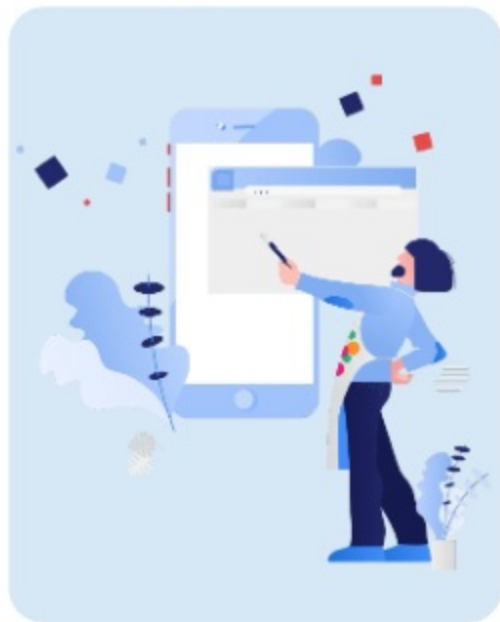
2nd-generation managed packages, or also known as **managed 2GP**, are used for distributing apps on **AppExchange** using **Salesforce DX**.

❖ UNLOCKED PACKAGES

Unlocked packages are related or organized metadata that are packaged together using the **package development model** to build apps that are **versionable**, **easy to maintain**, **update**, **install**, and **upgrade**.

❖ CODE COVERAGE

Code coverage details included in the report can be used to identify which **Apex classes** in the package fail to meet the required code coverage percentage.



Unlocked Packages

The **force:package:version:report** command is used to retrieve details of a **package version**. To include code coverage details, the **--verbose** flag can be added.

Terminal output showing the command and its results:

```
-MacBook-Pro dreamhouse-lwc % sfdx force:package:version:report -p 04t5g000000UkN1AAK -v @gmail.com --verbose
Using specified username @gmail.com

==== Package Version
```

Name	Value
Name	Version 1.0
Subscriber Package Version Id	04t5g000000UkN1AAK
Id	0515g000000PBd9AAG
Package Id	0Ho5g000000Gn7YCAS
Version	1.0.0.1
Description	
Branch	
Tag	
Released	false
Validation Skipped	false
Ancestor	N/A
Ancestor Version	N/A
Code Coverage	
Code Coverage Met	false
Converted From Version Id	
Org-Dependent Unlocked Package	No
Release Version	55.0
Build Duration in Seconds	104
Managed Metadata Removed	N/A
Dependencies	
Created By	0055g000000DKoTBAAL
Code Coverage Details	N/A

Annotations:

- Package version Id**: Points to the package version ID `04t5g000000UkN1AAK` in the command.
- Code coverage details of Apex classes in the package will be displayed at the end**: Points to the `Code Coverage Details` field in the output table.

Best Practices

Table of Contents



Best Practices for Apex Testing

The **following** are some **best practices** when testing Apex code:



UNIT TESTING

Unit Testing should include **valid** and **invalid inputs** and execute **each branch** of conditional logic.



TEST DATA

Test data should be set up using a **separate class**, and it should be created before **Test.startTest** method.



CODE COVERAGE

Every use case of the application should be **covered** by tests, and tests should be **rerun** to refresh code coverage.



PARALLEL TEST EXECUTION

Parallel Test Execution should be **disabled** in Setup to **avoid** data contention issues and other errors.

Unit Testing Best Practices

The following are best practices when **developing** and **executing** unit tests:



ANNOTATE TEST CLASSES

The **@isTest annotation** should be used for **every** test class.



USE ASSERT METHODS

Assert methods should be used to prove that **code behaves properly**.



INPUTS AND BRANCHES

Method calls should be made using both **valid** and **invalid inputs**, and **each branch** of **conditional logic** must be executed.



CATCH/HANDLE ERRORS

Expected errors should be caught in a **try-catch block**, and caught exceptions should always be handled.

Unit Testing Best Practices



USER CONTEXT

If required, the **System.runAs method** should be used to test code in **different user contexts**.



TRIGGER TESTS

Tests should be written to **handle bulk trigger functionality**, and all triggers must have **at least one line** of test coverage.



INDIVIDUAL TESTS

Each class in an application should be **tested individually**.

Test as Different Users

System.runAs enables code to run in the context of a **specified user**, enforcing the user's **record sharing, object-level** and **field-level permissions** to test code behavior and data security.

```
@isTest
public static void test2() {

    Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User'];
    User newUser = new User(Email='newuser@acme.com',UserName='newuser@acme.com.dev',LastName='Testing',Alias='newUser',
    ProfileId=p.Id,LanguageLocaleKey='en_US',LocaleSidKey='en_US',TimeZoneSidKey='America/Los_Angeles',EmailEncodingKey='UTF-8');

    System.runAs(newUser) {
        Assert.areEqual('newuser@acme.com.dev', UserInfo.getUserName());
        Assert.areEqual(p.Id, UserInfo.getProfileId());
    }

    User currentUser = [SELECT Id, ProfileId FROM User WHERE UserName='currentuser@acme.com.dev'];
    System.runAs(currentUser) {
        Assert.areEqual('currentuser@acme.com.dev', UserInfo.getUserName());
        Assert.areEqual(currentUser.ProfileId, UserInfo.getProfileId());
    }
}
```

← Execute code as *newUser*

← Execute code as *currentUser*

Best Practices for Setting Up Test Data

The following are some **considerations** and **best practices** related to test data:



DATA DEPENDENCY

Test data should be used to **reduce dependencies** on data in sandbox and production orgs.



DATA GENERATION

Test data should be created before calling the Test.startTest method.



TEST DATA FACTORY

Separate test classes should be used to set up the necessary data.



DATA DELETION

Deleting test data is **not necessary** since tests do not commit.

Best Practices for Setting Up Test Data

This Apex class **updates the Industry field** of accounts if the value of the **Type field is 'Reseller'**.

```
public class AccountClass {  
  
    public static List<Account> processAccount(List<Account> accounts) {  
  
        List<Account> updatedAccounts = new List<Account>();  
  
        For (Account a : accounts) {  
  
            if(a.Type == 'Reseller') {  
                a.Industry = 'Manufacturing';  
                updatedAccounts.add(a);  
            }  
        }  
  
        update updatedAccounts;  
        return updatedAccounts;  
    }  
}
```

Best Practices for Setting Up Test Data

This test class **uses a separate class to create test data** and an **assert method** to prove the value of an updated field.

```
@isTest
private class AccountTest {

    @isTest
    static void testAccountProcessing () {

        String type = 'Reseller';
        String industry = 'IT';

        // Utility class for creating test data
        List<Account> accs = TestDataFactory.createTestAccounts(type, industry);

        AccountClass.processAccount(accs);

        for (Account a : accs) {
            Assert.areEqual('Manufacturing', a.Industry); // validate fields
        }
    }
}
```

Best Practices for Setting Up Test Data

This utility class **creates account data** for testing.

```
@isTest
public class TestDataFactory {

    public static List<Account> createTestAccounts(String t, String i) {

        List<Account> accounts = new List<Account>();

        for (Integer j = 1; j <= 300; j++) {
            Account a = new Account(Name = 'Test' + j, Type = t, Industry = i);
            accounts.add(a);
        }

        insert accounts;
        return accounts;
    }
}
```

Test Coverage Best Practices

The following are some **considerations** and **best practices** related to code coverage requirements:



OPTIMIZE COVERAGE

Even though 75% of Apex code must be covered by unit tests, a developer should cover as many lines of code as possible while **ensuring that every use case of the application is covered.**



RERUN TESTS

Tests should be rerun to refresh code coverage numbers and get a correct estimate.



MANAGED PACKAGES

Managed package tests are **not included** in the overall code coverage percentage.

Test Coverage Best Practices



COVERAGE CALCULATION

One should be careful while adding new code lines, since **code coverage depends** on the **total number** of **code lines** in the org.



INSUFFICIENT COVERAGE

More unit tests should be written if a deployment to production **fails due** to **insufficient code coverage**.



INCREASE COVERAGE

Additional tests should be written for classes with **less than 75% coverage** in sandbox.

Best Practices for Parallel Test Execution

The following are some **considerations** and **best practices** related to parallel test execution:



PARALLEL EXECUTION

Tests executed from the **Salesforce user interface** and **Developer Console** are executed in parallel.



ISSUES AND ERRORS

Data contention issues and **UNABLE_TO_LOCK_ROW** errors can arise due to parallel test execution.



SAME RECORDS

During parallel test execution, **tests may update the same records** if they do not create their own test data.



TURNING OFF

Such errors can be avoided by **turning off parallel test execution** by navigating to 'Application Test Execution' in Setup, clicking 'Options', and selecting 'Disable Parallel Apex Testing'.

Other Best Practices for Testing

Other best practices related to testing include **adding comments** and **sorting records**.



ADD COMMENTS

Comments should be included in tests with assumptions and expected outcomes.



SORT RECORDS

The **ORDER BY keywords** should be used to ensure that records are returned in the expected order.

Learn More

-  [What Are Apex Unit Tests?](#)
-  [Get Started with Apex Unit Tests](#)
-  [Using Test Setup Methods](#)
-  [Common Test Utility Classes for Test Data Creation](#)
-  [Loading Test Data \[From Static Resource\]](#)
-  [Testing HTTP Callouts by Implementing the HttpCalloutMock Interface](#)
-  [Asynchronous Apex](#)



Learn More

-  [Test Web Service Callouts \[for Testing SOAP Callouts\]](#)
-  [Testing Best Practices](#)
-  [Code Coverage Best Practices](#)
-  [Test Your Code](#)
-  [Use Mocks and Stub Objects](#)
-  [Build a Mocking Framework with the Stub API](#)

